

DSA Algorithms

Complete Guide

All major Data Structures & Algorithms implemented in **JavaScript**

#	Category	Topics
1	1. Sorting	1.1 Bubble Sort 1.2 Merge Sort 1.3 Selection Sort 1.4 Insertion Sort 1.5 Quick Sort 1.6 Heap Sort 1.7 Counting Sort
2	2. Searching	2.1 Linear Search 2.2 Binary Search 2.3 Jump Search 2.4 Interpolation Search 2.5 Exponential Search 2.6 Ternary Search
3	3. Dynamic Programming	3.1 Fibonacci Sequence 3.2 0/1 Knapsack Problem 3.3 Longest Common Subsequence 3.4 Longest Increasing Subsequence 3.5 Matrix Chain Multiplication 3.6 Edit Distance
4	4. Graph	4.1 Depth First Search 4.2 Breadth First Search 4.3 Dijkstra's Algorithm 4.4 Bellman-Ford Algorithm 4.5 Floyd-Warshall Algorithm 4.6 A* Search Algorithm 4.7 Kruskal's Algorithm 4.8 Prim's Algorithm 4.9 Tarjan's SCC 4.10 Kosaraju's SCC 4.11 Cycle Detection
5	5. Greedy	5.1 Activity Selection Problem 5.2 Kruskal's Algorithm 5.3 Prim's Algorithm
6	6. Backtracking	6.1 N-Queens Problem 6.2 Sudoku Solver

7	7. Mathematical	7.1 Euclidean Algorithm (GCD) 7.2 Sieve of Eratosthenes 7.3 Modular Exponentiation
8	8. Tree	8.1 Tree Traversals 8.2 Binary Search Tree (BST) 8.3 Lowest Common Ancestor (LCA) 8.4 AVL Tree
9	9. Bit Manipulation	9.1 Counting Set Bits 9.2 Single Non-Duplicate Element 9.3 Common Bit Tricks
10	10. Other Algorithms	10.1 Topological Sorting 10.2 Union-Find (Disjoint Set Union)

Sorting Algorithms

1. Bubble Sort

Theory: Bubble Sort is a simple comparison-based sorting algorithm. It repeatedly compares adjacent elements and swaps them when they are in the wrong order.

Algorithm:

- Start from the first element and compare it with the next element.
- Swap the two values if the left value is greater than the right value.
- Continue across the array until one full pass is completed.
- Repeat the passes until no swaps are needed.

Quick Questions

Q1. What is the main idea behind 1. Bubble Sort?

A. The core idea of 1. Bubble Sort is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 1. Bubble Sort?

A. Use 1. Bubble Sort when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 1. Bubble Sort?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 1. Bubble Sort work correctly.

JavaScript

```
1  function bubbleSort(arr){
2  for(let i=0; i<arr.length-1; i++){
3  let swapped=false;
4  for(let j=0; j<arr.length-i-1; j++){
5  if(arr[j]>arr[j+1]){
6  [arr[j], arr[j+1]] = [arr[j+1], arr[j]];
7  swapped=true;
8  }
9  }
10 if(!swapped) break;
11 }
12 return arr;
13 }
```

2. Merge Sort

Theory: Merge Sort uses divide and conquer. It splits the array into smaller parts, sorts each part, and merges the sorted parts together.

Algorithm:

- Divide the array into two halves.
- Recursively sort the left half and the right half.

- Merge the two sorted halves into one sorted array.

Quick Questions

Q1. What is the main idea behind 2. Merge Sort?

A. The core idea of 2. Merge Sort is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 2. Merge Sort?

A. Use 2. Merge Sort when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 2. Merge Sort?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 2. Merge Sort work correctly.

JavaScript

```

1  function mergeSort(arr){
2  if(arr.length<=1)return arr;
3  const mid=Math.floor(arr.length/2);
4  return merge(mergeSort(arr.slice(0,mid)),mergeSort(arr.slice(mid)));
5  }
6
7  function merge(left,right){
8  const out=[];
9  let i=0,j=0;
10 while(i<left.length&&j<right.length){
11   out.push(left[i]<=right[j]?left[i++]:right[j++]);
12 }
13 return [...out,...left.slice(i),...right.slice(j)];
14 }

```

3. Selection Sort

Theory: Selection Sort repeatedly selects the smallest value from the unsorted part of the array and places it in the correct position.

Algorithm:

- Start from the first index and treat it as the current minimum.
- Scan the remaining unsorted elements to find the real minimum.
- Swap the minimum value into the current position.
- Move to the next position and repeat.

Quick Questions

Q1. What is the main idea behind 3. Selection Sort?

A. The core idea of 3. Selection Sort is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 3. Selection Sort?

A. Use 3. Selection Sort when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 3. Selection Sort?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 3. Selection Sort work correctly.

JavaScript

```
1  function selectionSort(arr){
2    for(let i=0; i<arr.length-1; i++){
3      let min=i;
4      for(let j=i+1; j<arr.length; j++){
5        if(arr[j]<arr[min]) min=j;
6      }
7      [arr[i], arr[min]] = [arr[min], arr[i]];
8    }
9    return arr;
10 }
```

4. Insertion Sort

Theory: Insertion Sort builds the sorted array one element at a time. Each new element is inserted into its correct position among the already sorted elements.

Algorithm:

- Start from the second element.
- Compare it with elements to its left.
- Shift larger elements one position to the right.
- Insert the current element into the correct place.

Quick Questions

Q1. What is the main idea behind 4. Insertion Sort?

A. The core idea of 4. Insertion Sort is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 4. Insertion Sort?

A. Use 4. Insertion Sort when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 4. Insertion Sort?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 4. Insertion Sort work correctly.

JavaScript

```

1  function insertionSort(arr){
2  for(let i=1;i<arr.length;i++){
3  const key=arr[i];
4  let j=i-1;
5  while(j>=0&&arr[j]>key){
6  arr[j+1]=arr[j];
7  j--;
8  }
9  arr[j+1]=key;
10 }
11 return arr;
12 }

```

5. Quick Sort

Theory: Quick Sort is a divide-and-conquer sorting algorithm. It chooses a pivot, partitions the array, and recursively sorts the left and right parts.

Algorithm:

- Choose a pivot element.
- Move smaller elements to the left side and larger elements to the right side.
- Put the pivot into its final position.
- Recursively sort the left and right partitions.

Quick Questions

Q1. What is the main idea behind 5. Quick Sort?

A. The core idea of 5. Quick Sort is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 5. Quick Sort?

A. Use 5. Quick Sort when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 5. Quick Sort?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 5. Quick Sort work correctly.

JavaScript

```

1  function quickSort(arr, low=0, high=arr.length-1){
2  if( low>=high) return arr;
3  const p=partition(arr, low, high);
4  quickSort(arr, low, p-1);
5  quickSort(arr, p+1, high);
6  return arr;
7  }
8
9  function partition(arr, low, high){
10 const pivot=arr[high];
11 let i=low;
12 for( let j=low; j<high; j++){
13 if(arr[j]<=pivot)[arr[i], arr[j]]=[arr[j], arr[i]], i++;
14 }
15 [arr[i], arr[high]]=[arr[high], arr[i]];
16 return i;
17 }

```

6. Heap Sort

Theory: Heap Sort uses a binary heap to repeatedly extract the largest element and place it at the end of the array.

Algorithm:

- Build a max heap from the array.
- Swap the root with the last element.
- Reduce the heap size and heapify again.
- Repeat until all elements are sorted.

Quick Questions

Q1. What is the main idea behind 6. Heap Sort?

A. The core idea of 6. Heap Sort is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 6. Heap Sort?

A. Use 6. Heap Sort when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 6. Heap Sort?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 6. Heap Sort work correctly.

JavaScript

```

1  function heapSort(arr){
2  const n=arr.length;
3  const heapify=(size,i)=>{
4  let largest=i;
5  const left=2*i+1, right=2*i+2;
6  if( left<size&&arr[left]>arr[largest]) largest=left;
7  if(right<size&&arr[right]>arr[largest]) largest=right;
8  if( largest!=i){
9  [arr[i],arr[largest]]=arr[largest],arr[i];
10 heapify(size, largest);
11 }
12 };
13 for( let i=Math.floor(n/2)-1;i>=0;i--) heapify(n,i);
14 for( let end=n-1;end>0;end--){
15 [arr[0],arr[end]]=arr[end],arr[0];
16 heapify(end, 0);
17 }
18 return arr;
19 }

```

7. Counting Sort

Theory: Counting Sort is a non-comparison sorting algorithm. It counts how many times each value appears and uses that count to rebuild the sorted array.

Algorithm:

- Find the range of values in the array.
- Create a count array and store the frequency of each value.
- Compute positions from the counts.
- Build the sorted output using the count data.

Quick Questions

Q1. What is the main idea behind 7. Counting Sort?

A. The core idea of 7. Counting Sort is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 7. Counting Sort?

A. Use 7. Counting Sort when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 7. Counting Sort?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 7. Counting Sort work correctly.

JavaScript


```
1  function countingSort(arr){
2  const max=Math.max(...arr);
3  const count=new Array(max+1).fill(0);
4  for(const num of arr) count[num]++;
5  const out=[];
6  for(let i=0;i<=max;i++){
7  while(count[i]-->0) out.push(i);
8  }
9  return out;
10 }
```

Searching Algorithms

1. Linear Search

Theory: Linear Search checks each element one by one until the target is found or the array ends.

Algorithm:

- Start from the first element.
- Compare each value with the target.
- Return the index when a match is found.
- Return -1 if the target is not present.

Quick Questions

Q1. What is the main idea behind 1. Linear Search?

A. The core idea of 1. Linear Search is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 1. Linear Search?

A. Use 1. Linear Search when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 1. Linear Search?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 1. Linear Search work correctly.

JavaScript

```
1  function linearSearch(arr, target){
2  for(let i=0; i<arr.length; i++){
3  if(arr[i]===target) return i;
4  }
5  return -1;
6  }
```

2. Binary Search

Theory: Binary Search works on sorted arrays. It keeps cutting the search space in half until the target is found or the range becomes empty.

Algorithm:

- Set low and high pointers for the search range.
- Find the middle index.
- If the middle value matches the target, return it.
- Otherwise move to the left half or right half and repeat.

Quick Questions

Q1. What is the main idea behind 2. Binary Search?

A. The core idea of 2. Binary Search is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 2. Binary Search?

A. Use 2. Binary Search when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 2. Binary Search?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 2. Binary Search work correctly.

JavaScript

```
1 function binarySearch(arr, target){
2   let left=0, right=arr.length-1;
3   while( left<=right){
4     const mid=left+Math.floor((right-left)/2);
5     if(arr[mid]===target)return mid;
6     if(arr[mid]<target)left=mid+1;
7     else right=mid-1;
8   }
9   return -1;
10 }
```

3. Jump Search

Theory: Jump Search works on sorted arrays. It jumps ahead by a fixed block size and then performs a short linear search inside the correct block.

Algorithm:

- Choose a block size, usually $\sqrt{n}$.
- Jump forward block by block until the target may be inside a block.
- Perform linear search inside that block.
- Return the index if found, otherwise return -1.

Quick Questions

Q1. What is the main idea behind 3. Jump Search?

A. The core idea of 3. Jump Search is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 3. Jump Search?

A. Use 3. Jump Search when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 3. Jump Search?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 3. Jump Search work correctly.

JavaScript

```

1  function jumpSearch(arr, target){
2      const n = arr.length;
3      const step = Math.floor(Math.sqrt(n));
4      let prev = 0, curr = step;
5      while (curr < n && arr[curr-1] < target) prev = curr, curr += step;
6      for (let i = prev; i < Math.min(curr, n); i++){
7          if (arr[i] === target) return i;
8      }
9      return -1;
10 }

```

4. Interpolation Search

Theory: Interpolation Search is useful for uniformly distributed sorted data. It estimates where the target should be instead of always checking the middle.

Algorithm:

- Use the values at low and high to estimate the likely position.
- Check the estimated position.
- Narrow the range left or right depending on the comparison.
- Repeat until found or the range becomes invalid.

Quick Questions

Q1. What is the main idea behind 4. Interpolation Search?

A. The core idea of 4. Interpolation Search is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 4. Interpolation Search?

A. Use 4. Interpolation Search when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 4. Interpolation Search?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 4. Interpolation Search work correctly.

JavaScript

```

1  function interpolationSearch(arr, target){
2      let low = 0, high = arr.length - 1;
3      while (low <= high && target >= arr[low] && target <= arr[high]){
4          const pos = low + Math.floor(((target - arr[low]) * (high - low)) / (arr[high]
...  - arr[low]));
5          if (arr[pos] === target) return pos;
6          if (arr[pos] < target) low = pos + 1;
7          else high = pos - 1;
8      }
9      return -1;
10 }

```

5. Exponential Search

Theory: Exponential Search is useful when the search range is not known in advance. It first expands the range quickly and then applies Binary Search.

Algorithm:

- Check the first element.
- Double the index until the value is greater than or equal to the target.
- Run Binary Search inside the discovered range.

Quick Questions

Q1. What is the main idea behind 5. Exponential Search?

A. The core idea of 5. Exponential Search is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 5. Exponential Search?

A. Use 5. Exponential Search when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 5. Exponential Search?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 5. Exponential Search work correctly.

JavaScript

```
1  functionexponentialSearch(arr,target){
2  if(arr[0]===target)return0;
3  leti=1;
4  while(i<arr.length&&arr[i]<=target)i*=2;
5  letleft=Math.floor(i/2),right=Math.min(i,arr.length-1);
6  while(left<=right){
7  constmid=left+Math.floor((right-left)/2);
8  if(arr[mid]===target)returnmid;
9  if(arr[mid]<target)left=mid+1;
10 else right=mid-1;
11 }
12 return-1;
13 }
```

6. Ternary Search

Theory: Ternary Search works on sorted arrays by dividing the search range into three parts instead of two.

Algorithm:

- Compute two middle points.
- Compare the target with both middle values.
- Continue searching in the left third, middle third, or right third.
- Repeat until found or the range is invalid.

Quick Questions

Q1. What is the main idea behind 6. Ternary Search?

A. The core idea of 6. Ternary Search is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 6. Ternary Search?

A. Use 6. Ternary Search when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 6. Ternary Search?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 6. Ternary Search work correctly.

JavaScript

```
1  functionternarySearch(arr, left, right, target){
2  if(right<left)return-1;
3  constmid1=left+Math.floor((right-left)/3);
4  constmid2=right-Math.floor((right-left)/3);
5  if(arr[mid1]===target)returnmid1;
6  if(arr[mid2]===target)returnmid2;
7  if(target<arr[mid1])returnternarySearch(arr, left,mid1-1,target);
8  if(target>arr[mid2])returnternarySearch(arr,mid2+1, right,target);
9  returnternarySearch(arr,mid1+1,mid2-1,target);
10 }
```

Dynamic Programming Algorithms

1. Fibonacci Sequence

Theory: Dynamic Programming stores previous Fibonacci values so the same subproblem is not solved again and again.

Algorithm:

- Start with the base cases 0 and 1.
- Store each next Fibonacci value as the sum of the previous two.
- Continue until the required index is reached.

Quick Questions

Q1. What is the main idea behind 1. Fibonacci Sequence?

A. The core idea of 1. Fibonacci Sequence is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 1. Fibonacci Sequence?

A. Use 1. Fibonacci Sequence when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 1. Fibonacci Sequence?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 1. Fibonacci Sequence work correctly.

JavaScript

```
1 function fibonacci(n){
2   if(n<=1) return n;
3   const dp=[0,1];
4   for(let i=2;i<=n;i++) dp[i]=dp[i-1]+dp[i-2];
5   return dp[n];
6 }
```

2. 0/1 Knapsack Problem

Theory: The 0/1 Knapsack problem chooses items to maximize value without crossing a weight limit. Dynamic Programming solves it by storing answers for smaller capacities.

Algorithm:

- Create a DP table for items and capacities.
- For each item, decide whether taking it gives a better value than skipping it.
- Fill the table row by row.
- Read the final answer from the last cell.

Quick Questions

Q1. What is the main idea behind 2. 0/1 Knapsack Problem?

A. The core idea of 2. 0/1 Knapsack Problem is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 2. 0/1 Knapsack Problem?

A. Use 2. 0/1 Knapsack Problem when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 2. 0/1 Knapsack Problem?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 2. 0/1 Knapsack Problem work correctly.

JavaScript

```
1 function knapsack(weights, values, capacity){
2   const n = values.length;
3   const dp = Array.from({length: n+1}, () => new Array(capacity+1).fill(0));
4   for(let i=1; i<=n; i++){
5     for(let w=1; w<=capacity; w++){
6       if(weights[i-1]<=w){
7         dp[i][w] = Math.max(values[i-1]+dp[i-1][w-weights[i-1]], dp[i-
... 1][w]);
8       } else dp[i][w] = dp[i-1][w];
9     }
10  }
11  return dp[n][capacity];
12 }
```

3. Longest Common Subsequence (LCS)

Theory: LCS finds the longest sequence that appears in the same order in two strings. A DP table stores the best answer for each prefix pair.

Algorithm:

- Build a 2D table for the two strings.
- If characters match, add 1 to the diagonal value.
- If they do not match, take the maximum of the left or top cell.
- The last cell gives the LCS length.

Quick Questions

Q1. What is the main idea behind 3. Longest Common Subsequence (LCS)?

A. The core idea of 3. Longest Common Subsequence (LCS) is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 3. Longest Common Subsequence (LCS)?

A. Use 3. Longest Common Subsequence (LCS) when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 3. Longest Common Subsequence (LCS)?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 3. Longest Common Subsequence (LCS) work correctly.

JavaScript


```

1  function lcs(a, b) {
2  const dp = Array.from({ length: a.length + 1 }, () => new Array(b.length + 1)
...  .fill(0));
3  for (let i = 1; i <= a.length; i++) {
4  for (let j = 1; j <= b.length; j++) {
5  dp[i][j] = a[i - 1] === b[j - 1] ? dp[i - 1][j - 1] + 1 : Math.max(dp[i - 1]
...  [j], dp[i][j - 1]);
6  }
7  }
8  return dp[a.length][b.length];
9  }

```

4. Longest Increasing Subsequence (LIS)

Theory: LIS finds the longest strictly increasing subsequence in an array. Dynamic Programming tracks the best increasing length ending at each index.

Algorithm:

- Initialize each position with length 1.
- Compare each element with all earlier elements.
- Extend the subsequence when the current value is larger.
- Keep the maximum length seen.

Quick Questions

Q1. What is the main idea behind 4. Longest Increasing Subsequence (LIS)?

A. The core idea of 4. Longest Increasing Subsequence (LIS) is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 4. Longest Increasing Subsequence (LIS)?

A. Use 4. Longest Increasing Subsequence (LIS) when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 4. Longest Increasing Subsequence (LIS)?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 4. Longest Increasing Subsequence (LIS) work correctly.

JavaScript

```

1  function lis(arr) {
2  const dp = new Array(arr.length).fill(1);
3  let best = 1;
4  for (let i = 1; i < arr.length; i++) {
5  for (let j = 0; j < i; j++) {
6  if (arr[i] > arr[j]) dp[i] = Math.max(dp[i], dp[j] + 1);
7  }
8  best = Math.max(best, dp[i]);
9  }
10 return best;
11 }

```

5. Matrix Chain Multiplication

Theory: Matrix Chain Multiplication finds the cheapest order to multiply matrices. Different parenthesizations can lead to very different costs.

Algorithm:

- Build a DP table for matrix ranges.
- Try every possible split point for each range.
- Store the minimum multiplication cost for that range.
- The full range stores the answer.

Quick Questions

Q1. What is the main idea behind 5. Matrix Chain Multiplication?

A. The core idea of 5. Matrix Chain Multiplication is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 5. Matrix Chain Multiplication?

A. Use 5. Matrix Chain Multiplication when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 5. Matrix Chain Multiplication?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 5. Matrix Chain Multiplication work correctly.

JavaScript

```
1  function matrixChainOrder(p){
2  const n=p.length;
3  const dp=Array.from({length:n},()=>new Array(n).fill(0));
4  for(let len=2; len<n; len++){
5  for(let i=1; i<n-len+1; i++){
6  const j=i+len-1;
7  dp[i][j]=Infinity;
8  for(let k=i; k<j; k++){
9  dp[i][j]=Math.min(dp[i][j], dp[i][k]+dp[k+1][j]+p[i-1]*p[k]*
...  p[j]);
10 }
11 }
12 }
13 return dp[1][n-1];
14 }
```

6. Edit Distance (Levenshtein Distance)

Theory: Edit Distance counts the minimum insertions, deletions, and replacements needed to convert one string into another.

Algorithm:

- Build a 2D DP table for the two strings.
- If characters are the same, copy the diagonal value.
- Otherwise take 1 plus the minimum of insert, delete, or replace.
- The last cell gives the final distance.

Quick Questions

Q1. What is the main idea behind 6. Edit Distance (Levenshtein Distance)?

A. The core idea of 6. Edit Distance (Levenshtein Distance) is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 6. Edit Distance (Levenshtein Distance)?

A. Use 6. Edit Distance (Levenshtein Distance) when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 6. Edit Distance (Levenshtein Distance)?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 6. Edit Distance (Levenshtein Distance) work correctly.

JavaScript

```
1  function editDistance(a, b){
2  const dp = Array.from({length: a.length+1}, (_, i) => Array.from({length:
...  b.length+1}, (_, j) => i+j));
3  for (let i = 1; i <= a.length; i++){
4  for (let j = 1; j <= b.length; j++){
5  if (a[i-1] === b[j-1]) dp[i][j] = dp[i-1][j-1];
6  else dp[i][j] = 1 + Math.min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]);
7  }
8  }
9  return dp[a.length][b.length];
10 }
```

Graph Algorithms

1. Depth First Search (DFS)

Theory: DFS explores one path as deeply as possible before backtracking. It uses recursion or an explicit stack.

Algorithm:

- Start from a source node and mark it visited.
- Visit one unvisited neighbor.
- Continue deeper until no unvisited neighbor remains.
- Backtrack and continue from earlier nodes.

Quick Questions

Q1. What is the main idea behind 1. Depth First Search (DFS)?

A. The core idea of 1. Depth First Search (DFS) is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 1. Depth First Search (DFS)?

A. Use 1. Depth First Search (DFS) when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 1. Depth First Search (DFS)?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 1. Depth First Search (DFS) work correctly.

JavaScript

```
1  function dfs(start, adj){
2    const visited = new Set();
3    const order = [];
4    function go(node){
5      visited.add(node);
6      order.push(node);
7      for(const next of adj[node] || []) if(!visited.has(next)) go(next);
8    }
9    go(start);
10   return order;
11 }
```

2. Breadth First Search (BFS)

Theory: BFS explores the graph level by level. It uses a queue to visit all neighbors before moving deeper.

Algorithm:

- Start from a source node and mark it visited.
- Push it into a queue.
- Remove one node at a time and add all unvisited neighbors.
- Continue until the queue is empty.

Quick Questions

Q1. What is the main idea behind 2. Breadth First Search (BFS)?

A. The core idea of 2. Breadth First Search (BFS) is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 2. Breadth First Search (BFS)?

A. Use 2. Breadth First Search (BFS) when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 2. Breadth First Search (BFS)?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 2. Breadth First Search (BFS) work correctly.

JavaScript

```
1  function bfs(start, adj){
2  const visited = new Set([start]);
3  const queue = [start], order = [];
4  while(queue.length){
5  const node = queue.shift();
6  order.push(node);
7  for(const next of adj[node] || []){
8  if(!visited.has(next)) visited.add(next), queue.push(next);
9  }
10 }
11 return order;
12 }
```

3. Dijkstra's Algorithm

Theory: Dijkstra's Algorithm finds shortest paths from one source in graphs with non-negative weights.

Algorithm:

- Set source distance to 0 and all others to infinity.
- Use a priority queue to pick the node with the smallest known distance.
- Relax all outgoing edges from that node.
- Repeat until all shortest distances are finalized.

Quick Questions

Q1. What is the main idea behind 3. Dijkstra's Algorithm?

A. The core idea of 3. Dijkstra's Algorithm is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 3. Dijkstra's Algorithm?

A. Use 3. Dijkstra's Algorithm when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 3. Dijkstra's Algorithm?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 3. Dijkstra's Algorithm work correctly.

JavaScript

```

1  functiondijkstra(src,V,adj){
2  constdist=newArray(V).fill(Infinity);
3  constused=newArray(V).fill(false);
4  dist[src]=0;
5  for(leti=0;i<V;i++){
6  letu=-1;
7  for(letv=0;v<V;v++)if(!used[v]&&(u===-1||dist[v]<dist[u]))
...  u=v;
8  if(u===-1)break;
9  used[u]=true;
10  for(const[to,w]of(adj[u]||[]))dist[to]=Math.min(dist[to],dist[u]+
...  w);
11  }
12  returndist;
13  }

```

4. Bellman-Ford Algorithm

Theory: Bellman-Ford finds shortest paths even when some edges have negative weights. It can also detect a negative cycle.

Algorithm:

- Initialize distances from the source.
- Relax every edge $V-1$ times.
- Run one more pass to check for a distance that still improves.
- If improvement happens, a negative cycle exists.

Quick Questions

Q1. What is the main idea behind 4. Bellman-Ford Algorithm?

A. The core idea of 4. Bellman-Ford Algorithm is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 4. Bellman-Ford Algorithm?

A. Use 4. Bellman-Ford Algorithm when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 4. Bellman-Ford Algorithm?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 4. Bellman-Ford Algorithm work correctly.

JavaScript

```

1  functionbellmanFord(V,edges,src){
2  constdist=newArray(V).fill(Infinity);
3  dist[src]=0;
4  for(leti=0;i<V-1;i++){
5  for(const[u,v,w]ofedges){
6  if(dist[u]!==Infinity&&dist[u]+w<dist[v])dist[v]=dist[u]+w;
7  }
8  }
9  returndist;
10 }

```

5. Floyd-Warshall Algorithm

Theory: Floyd-Warshall computes the shortest path between every pair of vertices in a graph.

Algorithm:

- Start with the direct-edge distance matrix.
- Pick each vertex as an intermediate point.
- Update every pair distance if going through that vertex is shorter.
- The final matrix contains all-pairs shortest paths.

Quick Questions

Q1. What is the main idea behind 5. Floyd-Warshall Algorithm?

A. The core idea of 5. Floyd-Warshall Algorithm is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 5. Floyd-Warshall Algorithm?

A. Use 5. Floyd-Warshall Algorithm when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 5. Floyd-Warshall Algorithm?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 5. Floyd-Warshall Algorithm work correctly.

JavaScript

```
1 function floydWarshall(graph){
2   const dist = graph.map(row => [...row]);
3   for(let k=0; k<dist.length; k++){
4     for(let i=0; i<dist.length; i++){
5       for(let j=0; j<dist.length; j++){
6         dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);
7       }
8     }
9   }
10  return dist;
11 }
```

6. A* Search Algorithm

Theory: A* is a pathfinding algorithm that combines actual travel cost with a heuristic estimate to the goal.

Algorithm:

- Put the start node into the open set.
- Always choose the node with the lowest $f = g + h$.
- Update neighbor costs and parents when a better path is found.
- Stop when the goal node is removed from the open set.

Quick Questions

Q1. What is the main idea behind 6. A* Search Algorithm?

A. The core idea of 6. A* Search Algorithm is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 6. A* Search Algorithm?

A. Use 6. A* Search Algorithm when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 6. A* Search Algorithm?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 6. A* Search Algorithm work correctly.

JavaScript

```
1 function aStar(start, goal, heuristic){
2   const open = [{node: start, g: 0, f: heuristic(start, goal)}];
3   while(open.length){
4     open.sort((a, b) => a.f - b.f);
5     const current = open.shift();
6     if(current.node === goal) return current.g;
7     for(const next of current.node.neighbors || []){
8       const g = current.g + next.cost;
9       open.push({node: next.node, g, f: g + heuristic(next.node, goal)});
10    }
11  }
12  return -1;
13 }
```

7. Kruskal's Algorithm

Theory: Kruskal's Algorithm builds a minimum spanning tree by choosing the lightest edges that do not create a cycle.

Algorithm:

- Sort all edges by weight.
- Pick the smallest edge.
- Add it only if it connects two different components.
- Continue until the spanning tree has $V-1$ edges.

Quick Questions

Q1. What is the main idea behind 7. Kruskal's Algorithm?

A. The core idea of 7. Kruskal's Algorithm is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 7. Kruskal's Algorithm?

A. Use 7. Kruskal's Algorithm when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 7. Kruskal's Algorithm?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 7. Kruskal's Algorithm work correctly.

JavaScript


```

1  functionkruskal(V,edges){
2  edges.sort((a,b)=>a[2]-b[2]);
3  constparent=Array.from({length:V}, (_, i)=>i);
4  constfind=x=>parent[x]==x?x:(parent[x]=find(parent[x]));
5  constmst=[];
6  for(const[u,v,w]ofedges){
7  constru=find(u),rv=find(v);
8  if(ru!==rv){
9  parent[ru]=rv;
10 mst.push([u,v,w]);
11 }
12 }
13 returnmst;
14 }

```

8. Prim's Algorithm

Theory: Prim's Algorithm grows a minimum spanning tree from one starting vertex by always choosing the cheapest connecting edge.

Algorithm:

- Start from any vertex.
- Add the smallest edge that connects the tree to a new vertex.
- Mark the new vertex as included.
- Repeat until all vertices join the tree.

Quick Questions

Q1. What is the main idea behind 8. Prim's Algorithm?

A. The core idea of 8. Prim's Algorithm is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 8. Prim's Algorithm?

A. Use 8. Prim's Algorithm when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 8. Prim's Algorithm?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 8. Prim's Algorithm work correctly.

JavaScript

```

1  function prim(V, adj){
2  const inMST=new Array(V).fill(false);
3  const key=new Array(V).fill(Infinity);
4  key[0]=0;
5  for(let count=0; count<V; count++){
6  let u=-1;
7  for(let v=0; v<V; v++) if(!inMST[v] && (u===-1 || key[v]<key[u])) u
...  =v;
8  inMST[u]=true;
9  for(const [to,w] of adj[u] || []) if(!inMST[to] && w<key[to]) key[to]
...  =w;
10 }
11 return key;
12 }

```

9. Tarjan's Algorithm (SCC)

Theory: Tarjan's Algorithm finds strongly connected components in a directed graph using DFS, discovery order, and low-link values.

Algorithm:

- Run DFS and assign discovery times.
- Track the lowest reachable discovery time for each node.
- Push active nodes onto a stack.
- When a node is the root of an SCC, pop its whole component.

Quick Questions

Q1. What is the main idea behind 9. Tarjan's Algorithm (SCC)?

A. The core idea of 9. Tarjan's Algorithm (SCC) is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 9. Tarjan's Algorithm (SCC)?

A. Use 9. Tarjan's Algorithm (SCC) when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 9. Tarjan's Algorithm (SCC)?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 9. Tarjan's Algorithm (SCC) work correctly.

JavaScript

```

1  function tarjanSCC(V, adj){
2      let time=0;
3      const disc=newArray(V).fill(-1), low=newArray(V).fill(0), stack=[],
...  inStack=newArray(V).fill(false), out=[];
4      function dfs(u){
5          disc[u]=low[u]=time++;
6          stack.push(u); inStack[u]=true;
7          for(const v of (adj[u] || [])){
8              if(disc[v]===-1) dfs(v), low[u]=Math.min(low[u], low[v]);
9              elseif(inStack[v]) low[u]=Math.min(low[u], disc[v]);
10         }
11         if(low[u]===disc[u]){
12             const comp=[];
13             while(true){
14                 const x=stack.pop(); inStack[x]=false; comp.push(x);
15                 if(x===u) break;
16             }
17             out.push(comp);
18         }
19     }
20     for(let i=0; i<V; i++) if(disc[i]===-1) dfs(i);
21     return out;
22 }

```

10. Kosaraju's Algorithm (SCC)

Theory: Kosaraju's Algorithm finds strongly connected components with two DFS passes.

Algorithm:

- Run DFS on the original graph and store finishing order.
- Reverse all graph edges.
- Run DFS in reverse finishing order on the reversed graph.
- Each DFS tree gives one SCC.

Quick Questions

Q1. What is the main idea behind 10. Kosaraju's Algorithm (SCC)?

A. The core idea of 10. Kosaraju's Algorithm (SCC) is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 10. Kosaraju's Algorithm (SCC)?

A. Use 10. Kosaraju's Algorithm (SCC) when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 10. Kosaraju's Algorithm (SCC)?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 10. Kosaraju's Algorithm (SCC) work correctly.

JavaScript

```

1  function kosaraju(V, adj){
2  const visited = new Array(V).fill(false), order = [];
3  function dfs(u){
4  visited[u] = true;
5  for(const v of adj[u] || []) if(!visited[v]) dfs(v);
6  order.push(u);
7  }
8  for(let i = 0; i < V; i++) if(!visited[i]) dfs(i);
9  const rev = Array.from({length: V}, () => []);
10 for(let u = 0; u < V; u++) for(const v of adj[u] || []) rev[v].push(u);
11 visited.fill(false);
12 return order.reverse();
13 }

```

11. Cycle Detection

Theory: Cycle Detection checks whether a graph contains a closed path. The approach depends on whether the graph is directed or undirected.

Algorithm:

- For an undirected graph, run DFS and track the parent node.
- For a directed graph, run DFS and track the recursion stack.
- If a back edge or repeated ancestor is found, a cycle exists.

Quick Questions

Q1. What is the main idea behind 11. Cycle Detection?

A. The core idea of 11. Cycle Detection is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 11. Cycle Detection?

A. Use 11. Cycle Detection when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 11. Cycle Detection?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 11. Cycle Detection work correctly.

JavaScript

```

1  function hasCycle(V, adj){
2  const visited = new Array(V).fill(false);
3  function dfs(node, parent){
4  visited[node] = true;
5  for(const next of adj[node] || []){
6  if(!visited[next]){if(dfs(next, node)) return true;}
7  elseif(next !== parent) return true;
8  }
9  return false;
10 }
11 for(let i = 0; i < V; i++) if(!visited[i] && dfs(i, -1)) return true;
12 return false;
13 }

```

Greedy Algorithms

1. Activity Selection Problem

Theory: Activity Selection chooses the maximum number of non-overlapping activities by always taking the next activity that finishes earliest.

Algorithm:

- Sort activities by finish time.
- Select the first activity.
- Select the next activity whose start time is not earlier than the last chosen finish time.
- Continue until all activities are processed.

Quick Questions

Q1. What is the main idea behind 1. Activity Selection Problem?

A. The core idea of 1. Activity Selection Problem is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 1. Activity Selection Problem?

A. Use 1. Activity Selection Problem when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 1. Activity Selection Problem?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 1. Activity Selection Problem work correctly.

JavaScript

```
1 function activitySelection(start, end) {
2   const ids = Array.from({length: start.length}, (_, i) => i).sort((a, b) =>
...   end[a] - end[b]);
3   const selected = [ids[0]];
4   let lastEnd = end[ids[0]];
5   for (let i = 1; i < ids.length; i++) {
6     if (start[ids[i]] >= lastEnd) selected.push(ids[i]), lastEnd = end[ids[i]];
7   }
8   return selected;
9 }
```

2. Kruskal's Algorithm

Theory: In greedy terms, Kruskal always takes the lightest safe edge available.

Algorithm:

- Sort edges by weight.
- Add the smallest edge that does not form a cycle.
- Repeat until the tree is complete.

Quick Questions

Q1. What is the main idea behind 2. Kruskal's Algorithm?

A. The core idea of 2. Kruskal's Algorithm is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 2. Kruskal's Algorithm?

A. Use 2. Kruskal's Algorithm when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 2. Kruskal's Algorithm?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 2. Kruskal's Algorithm work correctly.

JavaScript

```
1 functiongreedyKruskal(V,edges){
2   returnkruskal(V,edges);
3 }
```

3. Prim's Algorithm

Theory: In greedy terms, Prim always extends the current tree using the cheapest possible connecting edge.

Algorithm:

- Start with one vertex in the tree.
- Choose the minimum edge from the tree to an outside vertex.
- Add that vertex and edge.
- Continue until all vertices are included.

Quick Questions

Q1. What is the main idea behind 3. Prim's Algorithm?

A. The core idea of 3. Prim's Algorithm is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 3. Prim's Algorithm?

A. Use 3. Prim's Algorithm when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 3. Prim's Algorithm?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 3. Prim's Algorithm work correctly.

JavaScript

```
1 functiongreedyPrim(V,adj){
2   returnprim(V,adj);
3 }
```

Backtracking Algorithms

1. N-Queens Problem

Theory: The N-Queens problem places queens so that none attack each other. Backtracking tries one choice at a time and undoes bad choices.

Algorithm:

- Place a queen row by row.
- Check whether the current placement is safe.
- If safe, move to the next row.
- If no position works, backtrack to the previous row.

Quick Questions

Q1. What is the main idea behind 1. N-Queens Problem?

A. The core idea of 1. N-Queens Problem is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 1. N-Queens Problem?

A. Use 1. N-Queens Problem when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 1. N-Queens Problem?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 1. N-Queens Problem work correctly.

JavaScript

```
1  function solveNQueens(n){
2  const board = new Array(n).fill(-1), results = [];
3  function safe(r, c){
4  for(let i=0; i<r; i++){if(board[i]===c || Math.abs(board[i]-c)===
... Math.abs(i-r))return false;
5  return true;
6  }
7  function place(r){
8  if(r===n) return results.push([...board]);
9  for(let c=0; c<n; c++){if(safe(r, c)) board[r]=c, place(r+1),
... board[r]=-1;
10 }
11 place(0);
12 return results;
13 }
```

2. Sudoku Solver

Theory: A Sudoku solver fills empty cells while respecting row, column, and subgrid rules.

Algorithm:

- Find an empty cell.

- Try digits 1 through 9.
- Keep the digit only if the placement is valid.
- Recurse to the next empty cell, or backtrack if needed.

Quick Questions

Q1. What is the main idea behind 2. Sudoku Solver?

A. The core idea of 2. Sudoku Solver is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 2. Sudoku Solver?

A. Use 2. Sudoku Solver when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 2. Sudoku Solver?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 2. Sudoku Solver work correctly.

JavaScript

```

1  function solveSudoku(board){
2  function valid(r,c,num){
3  const br=3*Math.floor(r/3),bc=3*Math.floor(c/3);
4  for(let i=0;i<9;i++){
5  if(board[r][i]===num||board[i][c]===num||board[br+Math.floor(i/
... 3)][bc+(i%3)]===num)return false;
6  }
7  return true;
8  }
9  function solve(){
10 for(let r=0;r<9;r++)for(let c=0;c<9;c++)if(board[r][c]===0)
... {
11 for(let num=1;num<=9;num++)if(valid(r,c,num)){board[r][c]=
... num;if(solve())return true;}
12 board[r][c]=0;return false;
13 }
14 return true;
15 }
16 solve();
17 return board;
18 }
```


Mathematical Algorithms

1. Euclidean Algorithm (GCD)

Theory: The Euclidean Algorithm finds the greatest common divisor by repeatedly replacing the larger number with the remainder.

Algorithm:

- Start with two numbers a and b.
- While b is not 0, replace a with b and b with a % b.
- When b becomes 0, a is the GCD.

Quick Questions

Q1. What is the main idea behind 1. Euclidean Algorithm (GCD)?

A. The core idea of 1. Euclidean Algorithm (GCD) is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 1. Euclidean Algorithm (GCD)?

A. Use 1. Euclidean Algorithm (GCD) when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 1. Euclidean Algorithm (GCD)?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 1. Euclidean Algorithm (GCD) work correctly.

JavaScript

```
1 functiongcd(a, b){
2   while(b !== 0) [a, b] = [b, a % b];
3   return a;
4 }
```

2. Sieve of Eratosthenes

Theory: The Sieve of Eratosthenes finds all primes up to n by marking multiples of each prime.

Algorithm:

- Mark all numbers from 2 to n as prime at first.
- Start with 2.
- Mark all multiples of the current prime as non-prime.
- Move to the next unmarked number and repeat.

Quick Questions

Q1. What is the main idea behind 2. Sieve of Eratosthenes?

A. The core idea of 2. Sieve of Eratosthenes is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 2. Sieve of Eratosthenes?

A. Use 2. Sieve of Eratosthenes when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 2. Sieve of Eratosthenes?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 2. Sieve of Eratosthenes work correctly.

JavaScript

```
1  function sieve(n){
2    const isPrime = new Array(n+1).fill(true);
3    isPrime[0] = isPrime[1] = false;
4    for (let p = 2; p * p <= n; p++) if (isPrime[p]){
5      for (let i = p * p; i <= n; i += p) isPrime[i] = false;
6    }
7    return isPrime.flatMap((ok, i) => ok ? [i] : []);
8  }
```

3. Modular Exponentiation

Theory: Modular Exponentiation computes $(\text{base}^{\text{power}}) \% \text{mod}$ efficiently using repeated squaring.

Algorithm:

- If the exponent is 0, return 1.
- Repeatedly square the base while reducing with modulo.
- Multiply into the answer when the current exponent bit is 1.

Quick Questions

Q1. What is the main idea behind 3. Modular Exponentiation?

A. The core idea of 3. Modular Exponentiation is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 3. Modular Exponentiation?

A. Use 3. Modular Exponentiation when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 3. Modular Exponentiation?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 3. Modular Exponentiation work correctly.

JavaScript

```
1  function modPow(base, exp, mod){
2    let result = 1;
3    base %= mod;
4    while (exp > 0){
5      if (exp & 1) result = (result * base) % mod;
6      base = (base * base) % mod;
7      exp >>= 1;
8    }
9    return result;
10 }
```

Tree Algorithms

1. Tree Traversals

Theory: Tree traversals define the order in which nodes are visited.

Algorithm:

- In-order: left, root, right.
- Pre-order: root, left, right.
- Post-order: left, right, root.

Quick Questions

Q1. What is the main idea behind 1. Tree Traversals?

A. The core idea of 1. Tree Traversals is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 1. Tree Traversals?

A. Use 1. Tree Traversals when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 1. Tree Traversals?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 1. Tree Traversals work correctly.

JavaScript

```
1 function inorder(node, out=[]){
2   if(!node) return out;
3   inorder(node.left, out); out.push(node.val); inorder(node.right, out);
4   return out;
5 }
```

2. Binary Search Tree (BST)

Theory: A BST keeps smaller values on the left and larger values on the right, which makes search, insertion, and deletion efficient when the tree is balanced.

Algorithm:

- For insertion, move left or right until the correct empty position is found.
- For search, compare the key and move left or right until found or null.
- For deletion, handle leaf, one-child, and two-child cases.

Quick Questions

Q1. What is the main idea behind 2. Binary Search Tree (BST)?

A. The core idea of 2. Binary Search Tree (BST) is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 2. Binary Search Tree (BST)?

A. Use 2. Binary Search Tree (BST) when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 2. Binary Search Tree (BST)?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 2. Binary Search Tree (BST) work correctly.

JavaScript

```
1  classBST{
2  constructor(){this.root=null;}
3  insert(val,node=this.root){
4  if(!this.root)returnthis.root={val, left:null, right:null};
5  if(val<node.val)node.left?this.insert(val,node.left):node.left={
...  val, left:null, right:null};
6  elseif(val>node.val)node.right?this.insert(val,node.right):
...  node.right={val, left:null, right:null};
7  }
8  }
```

3. Lowest Common Ancestor (LCA)

Theory: LCA is the deepest node that is an ancestor of two target nodes.

Algorithm:

- If the current node matches one target, return it.
- Recursively search left and right subtrees.
- If both sides return a node, the current node is the LCA.
- Otherwise return the non-null side.

Quick Questions

Q1. What is the main idea behind 3. Lowest Common Ancestor (LCA)?

A. The core idea of 3. Lowest Common Ancestor (LCA) is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 3. Lowest Common Ancestor (LCA)?

A. Use 3. Lowest Common Ancestor (LCA) when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 3. Lowest Common Ancestor (LCA)?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 3. Lowest Common Ancestor (LCA) work correctly.

JavaScript

```
1  functionlca(node,a,b){
2  if(!node||node.val===a||node.val===b)returnnode;
3  constleft=lca(node.left,a,b),right=lca(node.right,a,b);
4  if(left&&right)returnnode;
5  returnleft||right;
6  }
```

4. AVL Tree

Theory: An AVL Tree is a self-balancing BST. After insertion or deletion, rotations keep the height difference small.

Algorithm:

- Insert or delete as in a normal BST.
- Update the node heights.
- Compute the balance factor.
- Perform LL, RR, LR, or RL rotation when needed.

Quick Questions

Q1. What is the main idea behind 4. AVL Tree?

A. The core idea of 4. AVL Tree is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 4. AVL Tree?

A. Use 4. AVL Tree when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 4. AVL Tree?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 4. AVL Tree work correctly.

JavaScript

```
1  classAVLNode{
2  constructor(key){this.key=key;this.left=null;this.right=null;
...  this.height=1;}
3  }
4
5  functionheight(node){returnnode?node.height:0;}
```

Bit Manipulation Algorithms

1. Counting Set Bits

Theory: Counting set bits means counting how many 1s appear in the binary form of a number.

Algorithm:

- Check the last bit with bitwise AND.
- Add it to the count.
- Right shift the number.
- Repeat until the number becomes 0.

Quick Questions

Q1. What is the main idea behind 1. Counting Set Bits?

A. The core idea of 1. Counting Set Bits is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 1. Counting Set Bits?

A. Use 1. Counting Set Bits when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 1. Counting Set Bits?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 1. Counting Set Bits work correctly.

JavaScript

```
1 functioncountSetBits(n){
2   letcount=0;
3   while(n>0)count+=n&1,n>>=1;
4   returncount;
5 }
```

2. Finding the Only Non-Duplicated Element

Theory: XOR is useful because equal numbers cancel each other out. After XOR-ing the whole array, the single unmatched value remains.

Algorithm:

- Start with result = 0.
- XOR every element with result.
- Return the final result.

Quick Questions

Q1. What is the main idea behind 2. Finding the Only Non-Duplicated Element?

A. The core idea of 2. Finding the Only Non-Duplicated Element is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 2. Finding the Only Non-Duplicated Element?

A. Use 2. Finding the Only Non-Duplicated Element when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 2. Finding the Only Non-Duplicated Element?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 2. Finding the Only Non-Duplicated Element work correctly.

JavaScript

```
1 function findSingle(nums){  
2   return nums.reduce((acc, num) => acc ^ num, 0);  
3 }
```

Other Algorithms

1. Topological Sorting

Theory: Topological Sorting orders the vertices of a DAG so every directed edge points from an earlier vertex to a later vertex.

Algorithm:

- Run DFS on the graph.
- Push each node after exploring its neighbors.
- Reverse the finishing order to get the topological order.

Quick Questions

Q1. What is the main idea behind 1. Topological Sorting?

A. The core idea of 1. Topological Sorting is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 1. Topological Sorting?

A. Use 1. Topological Sorting when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 1. Topological Sorting?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 1. Topological Sorting work correctly.

JavaScript

```
1  function topologicalSort(V, adj){
2  const visited = new Array(V).fill(false), stack = [];
3  function dfs(node){
4  visited[node] = true;
5  for(const next of adj[node] || []) if(!visited[next]) dfs(next);
6  stack.push(node);
7  }
8  for(let i = 0; i < V; i++) if(!visited[i]) dfs(i);
9  return stack.reverse();
10 }
```

2. Union-Find (Disjoint Set Union)

Theory: Union-Find groups elements into disjoint sets and supports fast merging and representative lookup.

Algorithm:

- Use find to locate the representative of a set.
- Use union to merge two sets.
- Apply path compression and union by rank for efficiency.

Quick Questions

Q1. What is the main idea behind 2. Union-Find (Disjoint Set Union)?

A. The core idea of 2. Union-Find (Disjoint Set Union) is to solve the problem in a predictable step-by-step way while improving correctness, efficiency, or both.

Q2. When should we use 2. Union-Find (Disjoint Set Union)?

A. Use 2. Union-Find (Disjoint Set Union) when the problem structure matches its strengths, such as its required input conditions, its time complexity, or its data-processing pattern.

Q3. What is the most important thing to remember about 2. Union-Find (Disjoint Set Union)?

A. The most important point is to understand its key rule, base condition, or invariant, because that is what makes 2. Union-Find (Disjoint Set Union) work correctly.

JavaScript

```
1  classUnionFind{
2  constructor(n){
3    this.parent=Array.from({length:n}, (_, i)=>i);
4    this.rank=newArray(n).fill(0);
5  }
6  find(x){
7    if(this.parent[x]!==x)this.parent[x]=this.find(this.parent[x]);
8    returnthis.parent[x];
9  }
10 union(a,b){
11   letra=this.find(a),rb=this.find(b);
12   if(ra===rb)returnfalse;
13   if(this.rank[ra]<this.rank[rb])[ra,rb]=[rb,ra];
14   this.parent[rb]=ra;
15   if(this.rank[ra]===this.rank[rb])this.rank[ra]++;
16   returntrue;
17 }
18 }
```